

# Database Systems I / Information Systems I

## Indexing — Complementary Material

Christian Rauch

(changed: 2026-06-29)

# Why Do We Need Indexing?

A **sequential scan** reads every record in a table to find a match – acceptable for small tables, but costly for large ones.

## Problem

Given a table with  $n$  records, finding a single row by primary key takes  $O(n)$  time with a full scan.

- ▶ Real databases hold millions to billions of records.
- ▶ Most queries filter on *one* key (e.g. `WHERE id = 42`).
- ▶ An **index** is a separate data structure that maps search-key values directly to record locations.
- ▶ Goal: answer equality lookups in  $O(1)$  expected time.

# The Big Picture – Index Types

Two classical index families:

| Structure            | Lookup          | Range queries        |
|----------------------|-----------------|----------------------|
| Hash table           | $O(1)$ expected | <i>not supported</i> |
| B <sup>+</sup> -Tree | $O(\log n)$     | <i>supported</i>     |

## When to use a hash index

Equality predicates only: WHERE id = ?

Not suitable for: WHERE salary > 50000

There are also more advanced index types (e.g. R-trees for spatial data).

# B<sup>+</sup> Tree – Structure

A **B<sup>+</sup> tree** is a balanced search tree optimised for block-oriented storage. Every path from root to leaf has the same length.

## Two node types

- ▶ **Internal nodes** – store  $n$  search keys and  $n+1$  child pointers; guide the search.
- ▶ **Leaf nodes** – store search keys together with *record pointers* (or the records themselves); linked in a doubly-linked list for range scans.

**Order  $t$ :** every non-root node holds between  $t$  and  $2t$  keys. This keeps the tree balanced and guarantees  $O(\log n)$  height.

Internal node:     $| p_0 | k_1 | p_1 | k_2 | p_2 | \cdots | k_n | p_n |$

Leaf node:         $| k_1, r_1 | k_2, r_2 | \cdots | k_n, r_n | \text{next} |$

# B<sup>+</sup> Tree – Lookup and Range Queries

**Point lookup** (find record with key  $k$ ):

1. Start at the root; at each internal node follow the child pointer  $p_i$  where  $k_i \leq k < k_{i+1}$ .
2. Repeat until a leaf is reached; scan the leaf for  $k$ .

**I/O cost:** one block read per level =  $O(\log_t n)$  I/Os.

With  $t \approx 100$  and  $10^8$  records:  $\leq 4$  I/Os.

**Range query** (e.g. WHERE salary BETWEEN 40k AND 60k)

1. Point-lookup the lower bound to reach the first matching leaf.
2. Follow the *leaf linked list* rightward until the upper bound is exceeded.

Cost:  $O(\log_t n + k/B)$  I/Os, where  $k$  is the result size and  $B$  records fit per block.

## B<sup>+</sup> Tree – Insertion

1. Point-lookup to find the target leaf  $L$ .
2. Insert the key–pointer pair into  $L$  in sorted order.
3. **If  $L$  is not full:** done. **2 I/Os** (read + write  $L$ ).
4. **If  $L$  overflows** ( $> 2t$  keys):
  - ▶ **Try rotation first** (see next slide).
  - ▶ If rotation is not possible: *split*  $L$  – allocate a new leaf  $L'$ , move the upper half of keys to  $L'$ , push the smallest key of  $L'$  up to the parent.
  - ▶ If the parent also overflows, split it too (propagates upward).
  - ▶ Only a root split increases tree height.

**I/O cost:**  $O(\log_t n)$  reads (path to leaf) +  $O(\log_t n)$  writes (split chain); splits are rare in practice.

## B<sup>+</sup> Tree – Rotation (Key Redistribution)

Before splitting an overflowing leaf  $L$ , check whether a **sibling** (adjacent leaf under the same parent) has a free slot. If so, *rotate* instead of splitting.

Rotation procedure (overflow in  $L$ , left sibling  $L_\ell$  has space)

1. Move the *smallest* key of  $L$  to the *end* of  $L_\ell$ .
2. Update the **separator key** in the parent to the new smallest key of  $L$ .

Symmetric for the right sibling.

Why prefer rotation over split?

- ▶ No new page is allocated – saves space and I/Os.
- ▶ Tree height is unchanged; no upward propagation.
- ▶ Cost: read/write  $L$  and  $L_\ell$  + write parent = **5 I/Os**, vs. a split (new page, possible cascade upward).
- ▶ Applies equally to **internal nodes** overflowing during a split propagation.

# B<sup>+</sup> Tree – Deletion

1. Point-lookup to find the leaf  $L$  containing the key.
2. Remove the key–pointer pair from  $L$ .
3. **If  $L$  still has  $\geq t$  keys:** done.
4. **If  $L$  underflows ( $< t$  keys):**
  - ▶ *Borrow* a key from a sibling (if sibling has  $> t$  keys).
  - ▶ Otherwise *merge*  $L$  with a sibling; remove the separator from the parent (may propagate upward).

**I/O cost:**  $O(\log_t n)$ ; merges are rare in practice.

## Contrast with hash tables

B<sup>+</sup> trees support both **equality** and **range** queries at  $O(\log_t n)$  I/Os. Hash tables answer equality in  $O(1)$  I/Os but cannot handle range queries at all.



# Hash Functions

A **hash function**  $h$  maps a search-key value  $k$  to a *bucket address*:

$$h : K \longrightarrow \{0, 1, \dots, B - 1\}$$

where  $B$  is the number of **buckets**.

## Desired properties

- ▶ **Uniform distribution** – keys spread evenly across buckets.
- ▶ **Deterministic** – same key always yields the same bucket.
- ▶ **Fast to compute** – ideally  $O(1)$  in key length.

## Classic example (integer keys):

$$h(k) = k \bmod B$$

Choose  $B$  to be a prime to improve distribution when keys are not random.

# Hash Functions – String Keys

For string (or composite) keys a common approach:

1. Convert each character to its numeric code  $c_i$ .
2. Accumulate:  $v = \sum_i c_i \cdot p^i \pmod{B}$  (polynomial rolling hash, prime  $p$ ).
3. Return  $v$  as the bucket index.

## Important

A hash function does *not* need to be injective. Multiple keys may map to the same bucket – this is a **collision** and must be handled explicitly.

# Buckets and Blocks

## Block (Page)

The unit of disk I/O. The OS and storage engine always read or write a full block at a time (typically 4 KB–16 KB). Reading a single record still costs **1 block I/O**.

## Bucket

A **logical** slot in the hash table – all records with  $h(k) = i$  belong to bucket  $i$ . In the simplest case a bucket maps to exactly **one block**.

| Bucket $i$ | Records stored         | Block on disk |
|------------|------------------------|---------------|
| 0          | A4 (id=40), A8 (id=80) | block $b_0$   |
| 1          | A1 (id=1)              | block $b_1$   |
| 2          | –                      | block $b_2$   |
| 3          | A3 (id=13), A6 (id=23) | block $b_3$   |

If a bucket holds more records than fit in one block, additional **overflow blocks** are chained to it (covered in Static Hashing).

## I/O Cost per Operation (One Block per Bucket)

Assumption: bucket  $i$  fits entirely in **one disk block**; no overflow.  
 $h(k)$  is computed in memory – **zero I/Os**.

| Operation | I/Os    | Why                                                                                           |
|-----------|---------|-----------------------------------------------------------------------------------------------|
| Lookup    | 1 read  | Compute $h(k)$ , read block $b_{h(k)}$ , scan records in memory.                              |
| Insertion | 2 (r+w) | Read block to check for duplicates and find free space, write block back with the new record. |
| Deletion  | 2 (r+w) | Read block to locate the record, write block back with the record removed.                    |

### With overflow chain (length $c$ )

Each operation must follow the chain: costs become  $1 + c$  reads for lookup/deletion and  $1 + c$  reads + 1 write for insertion/deletion.  
Overflow must be avoided to preserve  $O(1)$  I/O.

# Static Hashing

$B$  is fixed at creation time;  $h(k) = k \bmod B$  always maps to the same  $B$  primary buckets.

## Full Buckets and Overflow Buckets

When a primary bucket is full, an **overflow bucket** is allocated and chained to it:

$$[\text{primary}] \rightarrow [\text{overflow}_1] \rightarrow [\text{overflow}_2] \rightarrow \dots$$

- ▶ Each overflow bucket is a new page of the same fixed size.
- ▶ Lookup must follow the entire chain – cost grows with chain length.
- ▶ Many overflow buckets signal that  $B$  is too small.

**Rehashing:** when overflow becomes severe, rebuild the entire index with a larger  $B$  – requires a full scan of all records.

# Dynamic Hashing

$B$  grows (or shrinks) incrementally – no full rebuild required.

## Extendible Hashing

A *directory* of pointers doubles whenever a bucket overflows. Only the affected bucket is split; others are unaffected.

## Linear Hashing

Buckets split one at a time in a fixed round-robin order, regardless of which bucket actually overflowed. No directory needed.

Database systems typically prefer dynamic hashing to avoid the cost of periodic rehashing.

# Extendible Hashing – Structure

Uses a single hash function  $h$ ; only the first  $d$  bits of its output are used to index a **directory** of  $2^d$  pointers.  $d$  is the *global depth*; each bucket stores a *local depth*  $d_i$ .

## Where are the depths stored?

- ▶ **Global depth**  $d$ : one value in the directory header.
- ▶ **Local depth**  $d_i$ : one value in each bucket's header on disk.

When  $d_i < d$ , the bucket has not been split yet and  $2^{d-d_i}$  directory entries all point to it – lookup simply follows the pointer.

## One function, more bits over time

As the table grows,  $d$  increases – the *same*  $h$  is used, but more output bits are consulted. No second formula is needed.

Lookup: **2 I/Os** – 1 directory read (often cached), 1 bucket read.

# Extendible Hashing – Overflow Procedure

## When bucket $i$ is full:

1. If  $d_i < d$ : split bucket  $i$ , redistribute its records using bit  $d_i+1$ , increment  $d_i$ . Directory pointers are updated; no other buckets are touched.
2. If  $d_i = d$ : *double the directory* ( $d += 1$ ), then apply step 1. Each old entry  $j$  becomes entries  $j0$  and  $j1$ , both initially pointing to the same bucket – no bucket data moves.

## Why lookup never needs correction

Both operations preserve the invariant that every directory entry always points to the correct bucket: step 1 updates only the affected entries; step 2 copies pointers verbatim so nothing becomes stale. A wrong pointer can never arise – no correction code is needed.



## Linear Hashing – Idea

**Goal:** grow the hash table one bucket at a time – no directory, no full rebuild.

### Insertion and splitting are independent

- ▶ **Record placement:** always insert into the bucket the lookup rule selects for key  $k$  (using  $h_0$  or  $h_1$ ). If that bucket is full, attach a temporary overflow chain.
- ▶ **Split trigger:** when load factor  $\alpha$  exceeds a threshold, split **bucket**  $p$  (not the overflowing bucket) and advance  $p$  by one.

A full bucket that  $p$  has not yet reached keeps its overflow chain until  $p$  arrives and redistributes its records.

### Why split in fixed order?

Splitting an arbitrary bucket requires a directory to track which buckets have been split (that is extendible hashing). Fixed order reduces routing to one comparison:  $h_0(k) < p$ ?

## Linear Hashing – Two Hash Functions

Because some buckets have been split and others have not, two hash functions are active at the same time.

Let  $B_0$  be the **initial** number of buckets and  $p$  the current split pointer.

$$h_0(k) = k \bmod B_0 \quad h_1(k) = k \bmod 2B_0$$

Which function applies to which bucket?

- ▶ Buckets  $0 \dots p - 1$  have **already been split**. They now come in pairs: original bucket  $i$  and its new partner  $i + B_0$ . Keys in this range must be routed by  $h_1$ .
- ▶ Buckets  $p \dots B_0 - 1$  have **not yet been split**. Keys in this range are still routed by  $h_0$ .

After  $B_0$  splits,  $p$  wraps back to 0 with  $B_0 \leftarrow 2B_0$  and the cycle repeats with  $h_1$  and  $h_2 = k \bmod 4B_0$ , etc.

## Linear Hashing – Lookup

To find a record with search key  $k$ :

1. Compute  $b = h_0(k) = k \bmod B_0$ .
2. **Correction check:** is  $b < p$ ?
  - ▶ **Yes** – bucket  $b$  has already been split;  $h_0$  would send us to a bucket that no longer holds all the relevant keys. Use  $h_1(k) = k \bmod 2B_0$  instead to get the correct bucket.
  - ▶ **No** – bucket  $b$  has not been split yet;  $h_0$  is correct. Use  $b$  directly.
3. Read the chosen bucket (follow overflow chain if present).

### Cost

**1 I/O** in the common case (no overflow chain). The correction is arithmetic – zero extra I/Os.

### Contrast with extendible hashing

Extendible hashing stores the redirect in a directory (1 extra I/O or cache hit); linear hashing computes it with a single comparison. Linear hashing trades the directory for temporary overflow chains.

# Collisions

A **collision** occurs when two distinct keys  $k_1 \neq k_2$  satisfy  $h(k_1) = h(k_2)$ .

## Why collisions are unavoidable

By the *pigeonhole principle*: if  $|K| > B$ , at least one bucket must contain more than one key.

Two main strategies to resolve collisions:

1. **Chaining** (open hashing / closed addressing)
2. **Open addressing** (closed hashing / probing)

## Collision Resolution – Chaining

Each bucket holds a **linked list** of all records that hash to that address.

| Bucket | Chain           |
|--------|-----------------|
| 0      | [A4] → [A8] → ∅ |
| 1      | [A1] → ∅        |
| 3      | [A3] → [A6] → ∅ |

- ▶ **Insertion:** prepend to the chain –  $O(1)$ .
- ▶ **Lookup:** hash → scan chain –  $O(1)$  expected,  $O(n)$  worst case (all keys in one bucket).
- ▶ **Deletion:** find and unlink –  $O(1)$  expected.
- ▶ Memory overhead: one pointer per record.

## Collision Resolution – Open Addressing

All records are stored *inside* the bucket array. On collision, probe for the next free slot.

### Out-of-order placement

A record with key  $k$  is **not necessarily stored in bucket  $h(k)$**  – it ends up wherever the probe sequence finds a free slot. Lookup must *replay the same probe sequence* until it finds the record or hits an empty slot.

### Linear Probing

Try  $h(k) + 1, h(k) + 2, \dots$  (wrapping around):

$$h_i(k) = (h(k) + i) \bmod B, \quad i = 0, 1, 2, \dots$$

Disadvantage: *primary clustering* – long filled runs form near busy slots, slowing future probes.

## Open Addressing – Alternative Probe Sequences

To reduce primary clustering, use a probe sequence that spreads collisions more evenly.

### Quadratic Probing

$$h_i(k) = (h(k) + i^2) \bmod B$$

Reduces primary clustering but can cause *secondary clustering*: all keys hashing to the same slot follow the same sequence.

### Double Hashing

$$h_i(k) = (h(k) + i \cdot g(k)) \bmod B$$

where  $g$  is a second hash function. Each key gets its own step size – eliminates both clustering types.

Chaining avoids all clustering: every record sits in its home bucket (or overflow chain), never displaced to a neighbour.

# Load Factor and Performance

The **load factor**  $\alpha = n/B$  (records / buckets) governs performance.

| Strategy       | Avg. lookups ( $\alpha = 0.5$ ) | Avg. lookups ( $\alpha = 0.9$ ) |
|----------------|---------------------------------|---------------------------------|
| Chaining       | $\approx 1.25$                  | $\approx 1.45$                  |
| Linear probing | $\approx 1.5$                   | $\approx 5.5$                   |
| Double hashing | $\approx 1.4$                   | $\approx 2.6$                   |

## Rule of thumb

Keep  $\alpha \leq 0.7$  for open addressing; chaining tolerates higher load at the cost of pointer overhead.

The load factor also drives the decision to rehash (static) or split a bucket (dynamic hashing).